1046. Last Stone Weight

Easy · Heap / Priority Queue

leetcode.com/problems/last-stone-weight

Problem

- Repeatedly smash the two heaviest stones x <= y: if equal both vanish, else a stone of y - x remains. Return the last weight or 0.

Intuition

- We always need the two current maxima, so use a max-heap.

Brute Force

- Sort the array after every smash: O(n^2 log n).

Optimal Approach

- Push all stones into a max-heap.
- While size > 1: y = poll, x = poll; if y != x push y - x.
- Return heap empty ? 0 : peek.

Worked Example

```
[2,7,4,1,8,1].

8,7 -> 1   heap {4,2,1,1,1}
4,2 -> 2   heap {2,1,1,1}
2,1 -> 1   heap {1,1,1}
1,1 -> 0   heap {1}
answer 1
```

Java Solution

```java
public class LastStoneWeight {
    public int lastStoneWeight(int[] stones) {
        PriorityQueue<Integer> pq = new PriorityQueue<>(Collections.reverseOrder());
        for (int s : stones) pq.add(s);
        while (pq.size() > 1) {
            int y = pq.poll(), x = pq.poll();
            if (y != x) pq.add(y - x);
        }
        return pq.isEmpty() ? 0 : pq.peek();
    }
}
```

Complexity

- Time: O(n log n). Space: O(n).

Pitfalls

- Java PriorityQueue is a min-heap by default; pass Collections.reverseOrder().